Document 3. The Soar Models

M2, CLARION written in Soar (package `M2_clarion_in_soar`), and
M3, a native Soar model of self-control (package `M3_native_soar`)

Adan Gomez Salgado
SUNY Oswego

Version 2.0, 1 November 2026

Abstract

This document explains the two Soar models of the study. **M2** takes CLARION's equations and writes them as Soar rules; it checks whether the predictions of Document 2 belong to the theory rather than to one Python program. **M3** builds self-control from Soar's own mechanisms (reinforcement learning, deliberation in a substate, and chunking), without drives and without a cost of explicit processing; it is the real architectural contrast. For each model, the document describes every file of its package and how to use it, shows with the actual rules how the architecture appears in the code, follows one episode decision by decision, and analyzes the results hypothesis by hypothesis. Part I is a short introduction to Soar for readers who do not know it.

Contents

I Soar in brief **3**

1 What a reader needs to know about Soar **3**

2 How the Soar models talk to the robot **3**

II M2: Clarion written in Soar **4**

3 What M2 is for **4**

4 The package **4**

5 How Clarion becomes Soar **5**

5.1 One CLARION cycle as five Soar operators 5

5.2 Drives and value are inferences 5

5.3 The explicitness choice, exactly as in CLARION 6

5.4 Plan or habit 6

5.5 One episode 7

6 Results for M2 **8**

6.1 The three effects 8

6.2 Is M2 equivalent to M1? (H5) 8

6.3 Exploratory variants (run for this version) 9

6.4 What M2 shows 10

III M3: a native Soar model of self-control **10**

7 What M3 is for **10**

8 The package **10**

9 Design **11**

10 How the architecture appears in the code **11**

10.1 Behaviours are proposed when the world affords them 11

10.2 The habit: Soar-RL 11

10.3 Task knowledge marks conflicts 12

10.4 A conflict starts deliberation 12

10.5 The evaluation substate 13

10.6 Chunking 14

10.7 Capacity as time, in the bridge 14

11 Two episodes, decision by decision **16**

12 Results for M3 **17**

12.1 The hypotheses 17

12.2 Every variant 18

12.3 What M3 shows 18

13 Open design questions for M3 **18**

Part I
Soar in brief

1 What a reader needs to know about Soar

Soar (Laird, 2012) is a cognitive architecture built around a symbolic **working memory** and **production rules**.

- **Working memory** holds facts as triples, for example `(I2 ^battery-level high)`: object `I2` has attribute `battery-level` with value `high`. The robot's perception arrives on the `input-link`; commands to the robot are written on the `output-link`.
- **Rules** (written `sp {name conditions -> actions}`) fire when their conditions match working memory. Variables are written in angle brackets, like `<s>`.
- **Operators** are the actions Soar can select, including internal ones. In each **decision cycle**, rules *propose* operators, other rules attach *preferences* to them, the decision procedure *selects* one, and *apply* rules carry it out.
- **Preferences** are either symbolic (`>` best, `-` reject, `A > B`) or numeric (`= 0.7`). Symbolic preferences decide first. Among operators that only have numeric preferences, Soar can choose with the Boltzmann rule: probability proportional to $e^{v/T}$.
- **Support.** A fact created by an apply rule stays until it is removed (o-support). A fact created by any other rule disappears automatically when the rule's conditions stop matching (i-support). This is how Soar keeps inferences up to date.
- **Impasses and substates.** If Soar cannot choose (for example, two operators are equally best), or if the selected operator changes nothing, it creates a **substate**: a new context in which it can reason about the problem. Anything the substate adds to the state above it is a **result**; a result that resolves the impasse makes the substate disappear.
- **Chunking** turns the reasoning that produced a result into a new rule, so the same situation is handled directly the next time, without a substate.
- **Soar-RL** learns the numbers in numeric preferences from rewards, by reinforcement learning.

2 How the Soar models talk to the robot

Both models use the same protocol with the robot's world (file `soar_cleaner.py` for M2, `native_cleaner.py` for M3):

1. Python writes the perception to the input-link, with a cycle number `^cycle N`.
2. Python runs Soar one decision cycle at a time until a command for cycle N appears on the output-link.
3. Python reads the behaviour and executes it in the world for up to three steps.
4. At the next cycle, Soar removes the old command first.

The world, the task phases, the safety reflex and the battery cost of planning are exactly those of the CLARION robot (pyclarion-sc, bundled in both packages). Only the decision is made differently.

Part II
M2: Clarion written in Soar

3 What M2 is for

If CLARION's equations are written in Soar, Soar should behave like CLARION. Showing that it does is not a comparison of architectures. It is an *implementation check*: if the effects of Document 2 appear in a completely different program, they come from the equations of the theory, not from details of the Python code.

4 The package

File	What it is and when to use it
`clarion_in_soar/clarion_cleaner.soar`	The model: 64 Soar productions (Section 5).
`clarion_in_soar/soar_cleaner.py`	The bridge: builds the Soar agent, writes the perception and the parameters to the input-link, runs Soar until it answers, reads the command. A subclass of pyclarion-sc's robot, so the world and episode loop are shared.
`clarion_in_soar/variants.py`	The model and its variants: `clarion_in_soar`, `clarion_in_soar_mlp`, `M2_plan_only`, `M2_habit_only`, `M2_A1_fixed_drives`, `M2_A2_no_cost`.
`run_episode.py`, `run_study.py`, `verify.py`	The same generic scripts as in M1 (one episode with `-trace`; a full study; an exact re-run check).
`analyze.py`	The three effects; the equivalence test with M1 (H5); with `-variants`, the exploratory variants.
`common/`, `pyclarion_sc/`, `tests/test_m2.py`	Shared measures and statistics; bundled CLARION; four tests.
`results/grid_seeds_1-200.csv`	Confirmatory study (1,200 episodes).
`results/reference_M1_clarion_seeds_1-200.csv`	M1's episodes on the same seeds, for the equivalence test.
`results/new_variants_seeds_1-200.csv`	The exploratory variants (6,000 episodes, run for this version).
`results/grid_seeds_20001-20200.csv`, `reference_M1_...`	Replication.
`results/webots_*.csv`, `grid_seeds_30001-30100.csv`	Webots study and its grid twins.

```
cd M2_clarion_in_soar
pip install soar-sml                    # Soar 9.6.5 and its Python bindings
python -m unittest discover -s tests
```

```
python run_episode.py --variant clarion_in_soar --scenario anticipation --condition anticipation
    --seed 5001 --trace
python analyze.py --variants results/new_variants_seeds_1-200.csv
python verify.py
```

Reproducibility check. `verify.py` re-ran 18 stored episodes from the confirmatory file and 12 from the replication file; all were identical.

5 How Clarion becomes Soar

5.1 One Clarion cycle as five Soar operators

Soar operator	CLARION step	How it is done in Soar
`begin-cycle`	a new perception; time on task (Eq. 6)	clears the last command; updates `^time-on-task`
(no operator)	drive strength, goal strength, value (Eq. 3, 5, 7)	i-supported rules that compute the numbers on the right-hand side
`keep-goal`, `switch-goal`, `choose-goal`	goal choice with persistence	one `choose-goal` per goal, numeric preference gs_g/τ
`set-explicitness` × 21	explicitness (Eq. 2–4)	one operator per level j, numeric preference $j(\alpha\, value - c)/\tau$
`choose-level`	the plan acts with probability p_{explicit}	draws a random number; rules decide `^use top` or `bottom`
`plan-step` / `habit-step`	top / bottom level	one operator per applicable rule / per behaviour; the chosen one writes the command

5.2 Drives and value are inferences

The achievement stimulus depends on the phase (here, a choice ahead):

```
sp {ms*achievement*stimulus*ahead
   (state <s> ^clarion <c> ^io.input-link <il>)
   (<il> ^phase approaching-c ^params.stimulus-ahead <v>)
-->
   (<c> ^ach-stimulus <v>)
}
```

Drive strength (Eq. 5) and the value of the goal (Eq. 3) are computed on the right-hand side:

```
sp {ms*achievement*strength
   (state <s> ^clarion <c>)
   (<c> ^ach-stimulus <st> ^ach-deficit <d>)
-->
   (<c> ^ach-strength (* <st> <d>))
}
```

```
sp {mcs*value*clean
   (state <s> ^clarion <c>)
```

```
   (<c> ^goal clean_assigned ^ach-strength <a> ^clean-satisfaction <sat>)
-->
   (<c> ^value (* <a> <sat>))
}
```

Because these rules are i-supported, Soar recomputes the numbers automatically whenever the perception changes.

5.3 The explicitness choice, exactly as in Clarion

For each level j in working memory (0 to 20), one operator is proposed, carrying its utility divided by Clarion's temperature (line 9):

```
sp {propose*set-explicitness
   (state <s> ^clarion <c> ^io.input-link <il>)
   (<il> ^controller clarion ^busy false ^params <p>)
   (<p> ^alpha <alpha> ^cost <cost> ^temperature <tau>)
   (<c> ^stage explicitness ^value <v> ^level <j>)
-->
   (<s> ^operator <o> + =)
   (<o> ^name set-explicitness ^level <j> ^source utility
        ^u (/ (* <j> (- (* <alpha> <v>) <cost>)) <tau>))
}
```

A single shared rule turns that number into a numeric preference:

```
sp {all*numeric-preference
   (state <s> ^operator <o> +)
   (<o> ^u <u>)
-->
   (<s> ^operator <o> = <u>)
}
```

Clarion uses four temperatures (goal 0.1, explicitness 0.7, plan 0.1, habit 0.08); Soar has one. Setting Soar's temperature to 1 and dividing every preference by Clarion's temperature for that choice gives Soar's selection probability $e^{(U_j/\tau)/1} / \sum_k e^{(U_k/\tau)/1}$, which is exactly Clarion's rule. A test in the package checks this numerically (the mean drawn explicitness matches the formula within 0.03; in the original check, 0.610 against 0.608).

5.4 Plan or habit

`choose-level` draws a uniform random number, and an inference compares it with p_{explicit}:

```
sp {apply*choose-level
   (state <s> ^operator.name choose-level ^clarion <c>)
-->
   (<c> ^draw (rand-float 1.0) ^stage level - ^stage act)
}
```

```
sp {acs*use-top
   (state <s> ^clarion <c>)
   (<c> ^stage act ^p-explicit <p> ^draw < <p> ^applicable)
-->
   (<c> ^use top)
```

```
}
```

The plan's rules are inferences that mark themselves as applicable, for example rule P3:

```
sp {acs*top*P3
   (state <s> ^clarion <c> ^io.input-link <il>)
   (<c> ^goal clean_assigned)
   (<il> ^in-assigned-with-dirt false ^assigned-done false)
-->
   (<c> ^applicable <r>)
   (<r> ^rule P3 ^option next_room)
}
```

The habit's values are computed by rules from the same function that M1's network was trained to imitate. The temptation:

```
sp {acs*bottom*detour
   (state <s> ^clarion <c> ^io.input-link <il>)
   (<il> ^habit-mode rules ^temptation <t> ^temptation-distance <d>)
-->
   (<c> ^habit <x>)
   (<x> ^option detour ^q (* 0.95 (* <t> (- 1.0 (* 0.5 <d>)))))
}
```

5.5 One episode

t	phase	tempt.	ds	net	p_{expl}	Soar rule	plan says	chosen	by
57	before door	0.00	0.56	0.10	1.00	P3	next_room	next_room	top
60	before door	0.00	0.56	0.10	0.80	P3	next_room	next_room	top
63	before door	0.99	0.56	0.10	0.85	P3	next_room	next_room	top
66	after door	0.99	0.35	-0.01	0.95	P3	next_room	next_room	top
69	after door	0.99	0.35	-0.01	0.55	none	next_room	detour	bottom
72	after door	0.99	0.35	-0.01	0.00	none	next_room	detour	bottom
75	inside C	0.99	0.35	-0.01	0.05	none	next_room	detour	bottom
78	inside C	0.99	0.35	-0.01	1.00	P3	next_room	next_room	top
81	after door	0.99	0.35	-0.01	1.00	P3	next_room	next_room	top

Table 1: M2's decisions around room C's door (seed 5001, anticipation condition). *Soar rule*: the plan rule that chose, or *none* when the habit chose.

The numbers are the same as M1's at the same moments: drive 0.56 and net benefit +0.095 before the door, 0.35 and −0.010 after it. The individual random draws differ, because Soar has its own random number generator; that is why the two models' episodes are not identical, only statistically alike.

6 Results for M2

6.1 The three effects

Effect on lapse rate	M1 CLARION	M2 CLARION-in-Soar	M2 d_z	M2 p
anticipation (H1b)	0.198 [0.156, 0.242]	0.177 [0.127, 0.227]	0.48	<.001
motivational shift (H2)	0.171 [0.146, 0.195]	0.180 [0.155, 0.205]	0.99	<.001
capacity (H3)	0.269 [0.239, 0.299]	0.300 [0.269, 0.331]	1.30	<.001

Table 2: The three effects in M1 and M2 (seeds 1–200; mean difference in lapse rate, 95% CI).

All three effects appear in M2, with sizes close to M1's.

6.2 Is M2 equivalent to M1? (H5)

What equivalence means. "No significant difference" does not show that two things are the same: a small study can miss a real difference. An **equivalence test** (TOST, two one-sided tests) reverses the logic. One first decides how large a difference would matter (the *bounds*; here ±0.3 standard deviations of M1's measure), then tests whether the difference is reliably *inside* those bounds. In practice: equivalence is shown when the 90% confidence interval of the difference lies entirely between the bounds.

Condition	Measure	M1 mean	M2 mean	90% CI of M2 − M1	Bound	p (TOST)	Equivalent
anticipation/anticipation	lapse rate	0.35	0.33	[-0.05, 0.00]	±0.05	0.048	yes
anticipation/anticipation	room C (%)	11.65	10.64	[-3.88, 1.85]	±4.98	0.012	yes
anticipation/anticipation	steps	208.26	199.68	[-19.79, 2.61]	±20.61	0.039	yes
anticipation/anticipation	planning calls	41.38	39.64	[-3.47, 0.00]	±3.26	0.075	no
anticipation/constant	lapse rate	0.33	0.32	[-0.04, 0.01]	±0.04	0.034	yes
anticipation/constant	room C (%)	4.98	4.53	[-1.51, 0.61]	±2.12	0.005	yes
anticipation/constant	steps	184.49	180.50	[-8.18, 0.20]	±8.41	0.041	yes
anticipation/constant	planning calls	38.11	37.24	[-1.68, -0.05]	±1.52	0.091	no
capacity/busy	lapse rate	0.36	0.37	[-0.01, 0.04]	±0.04	0.032	yes
capacity/busy	room C (%)	10.30	11.00	[-1.91, 3.31]	±4.35	0.011	yes
capacity/busy	steps	203.19	202.84	[-11.52, 10.82]	±19.60	0.002	yes
capacity/busy	planning calls	38.09	37.45	[-2.28, 1.00]	±2.90	0.012	yes
capacity/free	lapse rate	0.23	0.23	[-0.03, 0.01]	±0.04	0.006	yes
capacity/free	room C (%)	1.78	2.07	[-0.29, 0.88]	±1.04	0.019	yes
capacity/free	steps	170.09	169.09	[-3.07, 1.05]	±4.85	0.001	yes
capacity/free	planning calls	41.66	41.16	[-1.06, 0.07]	±1.13	0.032	yes
shift/free	lapse rate	0.17	0.17	[-0.03, 0.01]	±0.04	0.005	yes
shift/free	room C (%)	0.77	0.89	[-0.21, 0.44]	±0.55	0.015	yes
shift/free	steps	164.56	163.32	[-2.69, 0.22]	±3.96	0.001	yes
shift/free	planning calls	44.71	44.57	[-0.60, 0.32]	±0.96	0.002	yes
shift/quiet	lapse rate	0.35	0.35	[-0.02, 0.02]	±0.04	0.002	yes
shift/quiet	room C (%)	10.41	13.35	[-0.20, 6.08]	±5.15	0.124	no
shift/quiet	steps	255.31	268.51	[-3.03, 29.44]	±28.96	0.055	no
shift/quiet	planning calls	56.24	58.22	[-0.69, 4.65]	±4.65	0.050	yes

Table 3: The 24 equivalence tests (seeds 1–200). Bound: ±0.3 SD of M1.

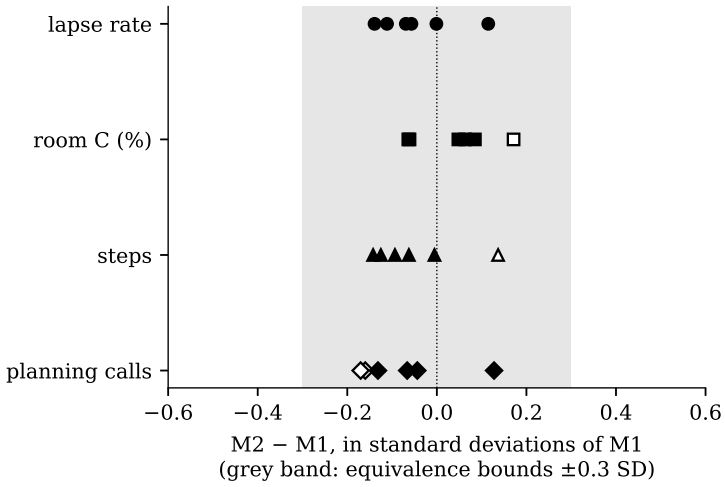


Figure 1: Each point is one of the 24 tests: the difference M2 − M1 in units of M1's standard deviation. Grey band: the equivalence bounds. Filled symbols: equivalence shown; open symbols: not shown.

Result. 20 of 24 tests pass; H5, as preregistered (all 24 must pass), is not supported. **How to read Figure 1.** Every difference is small (the largest is 0.17 SD). The four tests that fail are not large differences. Their confidence intervals simply reach slightly past the bound.

Why the failures do not indicate a real difference. If M2 and M1 were *exactly* equivalent, each of these tests would still fail sometimes, because 200 seeds give a confidence interval of limited width. Computed from the spread of the observed differences, the probability that a single test succeeds under exact equivalence (its power) ranges from 0.71 to 1.0 across the 24 tests, 0.84 on average. The expected number of failures under exact equivalence is therefore about 4. Finding 4 failures (and 1 on the replication seeds) is what equivalence predicts. The conjunction criterion of Document 1 was too strict to be informative; this is a design flaw of the protocol, and should be stated as such in the paper. A better criterion, for a future protocol, is equivalence on each measure pooled over conditions, or a correction that accounts for the number of tests.

6.3 Exploratory variants (run for this version)

Variant	Lapse rate	Anticipation (H1b)	Shift (H2)	Capacity (H3)	Steps	Planning calls
M2, habit as rules	0.292	0.177 [0.126, 0.227]	0.180 [0.155, 0.205]	0.300 [0.268, 0.331]	197	43.0
M2, habit from CLARION's network	0.296	0.172 [0.122, 0.223]	0.173 [0.147, 0.198]	0.293 [0.261, 0.325]	197	43.1
baseline: plan only	0.000	0.000 [0.000, 0.000]	0.000 [0.000, 0.000]	0.000 [0.000, 0.000]	167	56.5
baseline: habit only	0.994	0.000 [0.000, 0.000]	0.000 [0.000, 0.000]	0.000 [0.000, 0.000]	337	0.0
A1 fixed drives	0.238	0.000 [0.000, 0.000]	0.008 [-0.012, 0.029]	0.300 [0.268, 0.331]	183	43.3
A2 no cost	0.134	0.042 [0.023, 0.064]	0.080 [0.059, 0.102]	0.290 [0.258, 0.322]	175	49.1

Table 4: M2 and its exploratory variants (seeds 1–200). Effects: mean difference in lapse rate, 95% CI.

- **Habit from Clarion's network.** M2 using the network's values instead of the rule version passes 22 of 24 equivalence tests with M1. An earlier, smaller check (three conditions, 12 tests) had passed all 12, and the earlier report stated that this version "passes"; with all six conditions, two tests fail narrowly ($p = .051$ and .081). The corrected conclusion: using the

same habit makes M2 slightly closer to M1, but the remaining failures are of the size expected by chance (previous paragraph).

- **Baselines.** Plan only: no lapses; habit only: lapses at 99% of exposures. As in M1.
- **A1, fixed drives, and A2, no cost.** The same pattern as in M1: fixed drives remove the anticipation and shift effects and keep the capacity effect; no cost reduces lapses to about 0.13 and shrinks the two motivational effects. These ablations change only the parameters Soar reads from the input-link, not any rule.

6.4 What M2 shows

CLARION's mechanism, written in a different architecture and a different language, produces the same three effects, of similar size, and responds to the same ablations in the same way. The predictions belong to the framework, not to one program.

Part III
M3: a native Soar model of self-control

7 What M3 is for

M3 asks how Soar itself, with its own mechanisms, would produce self-control in this task, without borrowing CLARION's drives or its cost of explicit processing. Comparing M3 with M1 shows which architectural commitments are needed for which self-control effect. M3 is a first version (v0), written to be a reasonable Soar model, and will be revised with a student who works in Soar.

8 The package

File	What it is and when to use it
`native_soar/native_cleaner.soar`	The model: 45 Soar productions (Section 10).
`native_soar/habit_rl.soar`	The trained habit: 41 Soar-RL rules with their learned values.
`native_soar/native_cleaner.py`	The bridge: discretizes the perception into symbols, writes the reward, enforces the time budget (capacity scenario), reads the command and the reports. Also `train_habit()`.
`native_soar/variants.py`	`native_soar`, `native_habit_only`, `native_always_deliberate`, `N1_no_chunking`.
`train_habit.py`	Re-trains the habit, or shows its values (`-show`).
`run_episode.py`, `run_study.py`, `verify.py`, `analyze.py`	As in the other packages; `analyze.py` computes H4a–H4d.
`results/`	Confirmatory study (4,800 episodes), replication, Webots study and its grid twins.

```
cd M3_native_soar
python run_episode.py --variant native_soar --scenario capacity --condition busy --seed 5001 --
    trace
python run_episode.py --variant N1_no_chunking --scenario capacity --condition busy --seed 5001 --
    trace
python train_habit.py --show native_soar/habit_rl.soar
python analyze.py
python verify.py
```

Reproducibility check. `verify.py` re-ran 72 stored episodes (four variants, six conditions, three seeds) and 36 from the replication file; all were identical.

9 Design

1. Only Soar mechanisms. No drives, no explicitness utility, no cost term.
2. The temptation is *proposed* like any other option.
3. The habit is learned by Soar-RL from a reward (dirt cleaned in any room) that does not know which rooms are assigned.
4. Explicit control is *deliberation*: triggered by a conflict, carried out in a substate, costing decision cycles, and compiled by chunking.
5. Capacity is *time*: the number of decision cycles available before the robot must act.

10 How the architecture appears in the code

10.1 Behaviours are proposed when the world affords them

```
sp {propose*detour
   (state <s> ^stage behave ^io.input-link.temptation-level {<> none})
-->
   (<s> ^operator <o> +)
   (<o> ^name detour ^behaviour true)
}
```

10.2 The habit: Soar-RL

One template rule creates a numeric-preference rule for each combination of situation and behaviour that the robot meets:

```
sp {rl*habit
   :template
   (state <s> ^stage behave ^io.input-link <il> ^operator <o> +)
   (<il> ^in-assigned-with-dirt <a> ^quiet-zone <q> ^temptation-level <t>
         ^temptation-dist <d> ^battery-level <b> ^assigned-done <x>)
   (<o> ^name <n> ^behaviour true)
-->
   (<s> ^operator <o> = 0.0)
```

```
}
```

The bridge turns the continuous perception into these symbols (temptation none / low / high; near / far; battery low / mid / high). The habit was trained for 300 episodes, with reward = dirt cleaned since the last decision, in any room, and no deliberation. Its learned values:

Situation (battery high)	clean quiet	clean full	next room	detour	dock
corridor, room C in view (far)	–	–	0.11	5.79	0.00
corridor, room C in view (near)	–	–	0.38	5.63	0.01
corridor, nothing in view	–	–	2.94	–	0.88
dirty assigned room, quiet zone	0.17	2.90	–	–	0.00
dirty assigned room, no quiet zone	0.39	3.29	–	–	0.09

Table 5: Learned Soar-RL values (- = behaviour not offered in that situation). The higher the value, the more the habit prefers the behaviour (at test, Boltzmann with temperature 0.1).

With room C in view, the habit values the detour at about 5.7 and going to the next room at 0.1–0.4: an even stronger temptation than Clarion's habit has, and one learned from experience rather than imitated. In a quiet zone it prefers full suction (2.90 against 0.17): it has no notion of the constraint.

10.3 Task knowledge marks conflicts

```
sp {task*off-task*detour
   (state <s> ^operator <o> +)
   (<o> ^name detour)
-->
   (<s> ^conflict <c>)
   (<c> ^operator detour ^kind off-task)
}
```

Four such rules mark options that are off-task (the detour; docking early with work left), break a constraint (full suction in a quiet zone), or are risky (anything but docking on low battery). One flag summarizes them:

```
sp {elaborate*conflict-present
   (state <s> ^conflict <c>)
-->
   (<s> ^conflict-present true)
}
```

10.4 A conflict starts deliberation

```
sp {propose*deliberate
   (state <s> ^stage behave ^conflict-present true -^control
              ^io.input-link <il>)
   (<il> ^time-up false ^mode << native always >>)
-->
   (<s> ^operator <o> + >)
   (<o> ^name deliberate)
```

```
}
```

`deliberate` is proposed as the best operator when there is a conflict, no control knowledge yet (`-^control`), and time left (`^time-up false`). It has no application that solves the problem, so after it is selected Soar creates an **operator no-change substate**, the *evaluation space*.

10.5 The evaluation substate

Each candidate behaviour (each operator proposed in the state above) gets an `evaluate` operator, one per decision cycle:

```
sp {eval*propose*evaluate
   (state <ss> ^candidate <n> -^evaluation.operator <n>)
-->
   (<ss> ^operator <o> + =)
   (<o> ^name evaluate ^candidate <n>)
}
```

Its application gives the candidate a task value; for example, the detour is worth -1 and full suction in a quiet zone is worth -1:

```
sp {eval*apply*detour
   (state <ss> ^operator <o>)
   (<o> ^name evaluate ^candidate detour)
-->
   (<ss> ^evaluation <e>) (<e> ^operator detour ^value -1.0)
}
```

```
sp {eval*apply*clean-full*quiet
   (state <ss> ^operator <o> ^superstate.io.input-link.quiet-zone true)
   (<o> ^name evaluate ^candidate clean_full)
-->
   (<ss> ^evaluation <e>) (<e> ^operator clean_full ^value -1.0)
}
```

When every candidate has a value, `summarize` builds the control knowledge: reject every option with a negative value, prefer the best positive one:

```
sp {eval*apply*summarize*reject
   (state <ss> ^operator.name summarize ^summary <c> ^evaluation <e>)
   (<e> ^operator <n> ^value < 0.0)
-->
   (<c> ^reject <n>)
}
```

```
sp {eval*apply*summarize*prefer
   (state <ss> ^operator.name summarize ^summary <c> ^evaluation <e>)
   (<e> ^operator <n> ^value {<v> > 0.0})
  -{(<ss> ^evaluation <e2>)
    (<e2> ^value > <v>)}
  -(<ss> ^risky <n>)
-->
   (<c> ^prefer <n>)
}
```

and `conclude` hands it to the top state in one step, which resolves the impasse:

```
sp {eval*apply*conclude
   (state <ss> ^operator.name conclude ^superstate <s> ^summary <c>)
-->
   (<s> ^control <c>)
}
```

In the top state, the control knowledge becomes symbolic preferences, which override the habit's numbers:

```
sp {control*reject
   (state <s> ^control.reject <n> ^operator <o> +)
   (<o> ^name <n>)
-->
   (<s> ^operator <o> -)
}
```

```
sp {control*prefer
   (state <s> ^control.prefer <n> ^operator <o> +)
   (<o> ^name <n>)
-->
   (<s> ^operator <o> >)
}
```

10.6 Chunking

With chunking on, Soar compiles each deliberation into a rule. The first conflict in the corridor produced this chunk (printed by Soar):

```
sp {chunk*eval*apply*conclude*t9-1
    :chunk
    (state <s1> ^operator <o1> + ^operator <o2> + ^operator <o3> +
          ^operator <o4> ^io <i1>)
    (<o1> ^behaviour true ^name next_room)
    (<o2> ^behaviour true ^name detour)
    (<o3> ^behaviour true ^name dock)
    (<o4> ^name deliberate)
    (<i1> ^input-link <i2>)
    (<i2> ^assigned-done false ^battery-level { <b1> << mid high >> })
    -->
    (<s1> ^control <c1>)
    (<c1> ^from deliberation ^reject dock ^reject detour ^prefer next_room)}
```

In words: *when next room, detour and dock are on offer, rooms remain and the battery is not low, reject the dock and the detour and prefer the next room.* The first time, this took 9–11 decision cycles; afterwards, the chunk fires as soon as `deliberate` is selected, and a robot decision takes 4 cycles.

10.7 Capacity as time, in the bridge

```
def decide(self, p, phase):
    if p["battery"] < self.cfg.reflex_battery:
        return {"option": "dock", "level": "reflex", "reflex": True}
```

```
    a = self.agent
    self.n += 1
    cleaned = self.world.cleaned["assigned"] + self.world.cleaned["unassigned"]
    a.Update(self.w_reward, float(cleaned - self._cleaned_before))
    self._cleaned_before = cleaned
    for k, v in self.features(p).items():
        a.Update(self.w[k], v)
    a.Update(self.w["time-up"], "false")
    a.Update(self.w_cycle, self.n)
    a.Commit()
    budget = self.budget(p)
    vals, used, timed_out = None, 0, False
    while used < self.MAX_DECISIONS:
        if used >= budget and not timed_out:
            a.Update(self.w["time-up"], "true")
            a.Commit()
            timed_out = True
        a.RunSelf(1)
        used += 1
        vals = self._read()
        if vals:
            break
    if vals is None:
        raise RuntimeError("no command\n" + a.ExecuteCommandLine("print --stack"))
    deliberated = vals.get("deliberated") == "true"
    controlled = vals.get("controlled") == "true"
    if deliberated:
        self.world.charge_plan()            # same battery cost of explicit processing as CLARION
    return {
        "option": vals["option"], "level": "top" if controlled else "bottom", "reflex": False,
        "goal": "clean_assigned", "p_explicit": float(controlled), "explicitness_source": "native
    ",
        "explicitness_level": -1, "value": float("nan"), "drives": {}, "q": {},
        "top_rule": None, "bottom_option": None, "conflict": deliberated,
        "decisions_used": used, "budget": budget, "timed_out": timed_out and not controlled,
        "deliberated": deliberated,
    }
```

Lines 15–25: Soar runs one decision cycle at a time. When the number of cycles used reaches the budget, the bridge sets `^time-up true`; from then on `deliberate` is no longer proposed, an unfinished substate disappears, and the habit chooses. Control knowledge that already exists (from a chunk that has fired, or a deliberation that has just concluded) still applies. Budgets: 3–8 cycles while the computer is busy, 12–16 while it is free, no limit outside the capacity scenario.

11 Two episodes, decision by decision

t	phase	tempt.	delib.	cycles	budget	time-out	chosen	chosen by
54	travelling	0.00	True	4	6	False	next_room	control knowledge
57	before door	0.00	True	4	7	False	next_room	control knowledge
60	before door	0.00	True	4	7	False	next_room	control knowledge
63	before door	0.99	True	4	3	False	next_room	control knowledge
66	after door	0.99	True	4	6	False	next_room	control knowledge
69	after door	0.99	True	4	6	False	next_room	control knowledge
72	after door	0.00	True	4	7	False	next_room	control knowledge
75	after door	0.00	True	4	7	False	next_room	control knowledge
78	room B	0.00	True	4	7	False	clean_full	control knowledge

Table 6: M3 (chunking on), capacity scenario, busy computer, seed 5001. *cycles*: Soar decision cycles used for this robot decision; *budget*: cycles available.

t	phase	tempt.	delib.	cycles	budget	time-out	chosen	chosen by
54	travelling	0.00	True	8	6	True	next_room	habit (Soar-RL)
57	before door	0.00	True	9	7	False	next_room	control knowledge
60	before door	0.00	True	9	7	False	next_room	control knowledge
63	before door	0.99	True	5	3	True	detour	habit (Soar-RL)
66	before door	0.99	True	8	6	True	detour	habit (Soar-RL)
69	inside C	0.99	True	8	6	True	detour	habit (Soar-RL)
72	inside C	0.99	True	9	7	True	detour	habit (Soar-RL)
75	inside C	0.99	True	9	7	True	detour	habit (Soar-RL)
78	inside C	0.99	True	9	7	True	detour	habit (Soar-RL)

Table 7: The same seed with chunking off (N1).

How to read Tables 6 and 7.

- There is a conflict at almost every decision, even when room C is not in view: docking is always offered, and docking with work left is off-task. So M3 deliberates nearly all the time. With chunking, these deliberations are compiled after the first few and cost 4 cycles each.
- **With chunking (M3)**, the busy computer does not matter. At $t = 63$ the budget is only 3 cycles, yet control holds: the chunk has already fired, so the control knowledge exists before time runs out. The robot never detours.
- **Without chunking (N1)**, every deliberation must be done in full (8–9 cycles). At $t = 63$, room C comes into view and the budget is 3: the deliberation is cut short, the habit chooses, and the robot detours. Once inside room C, every deliberation is again cut short, so the habit keeps cleaning room C.

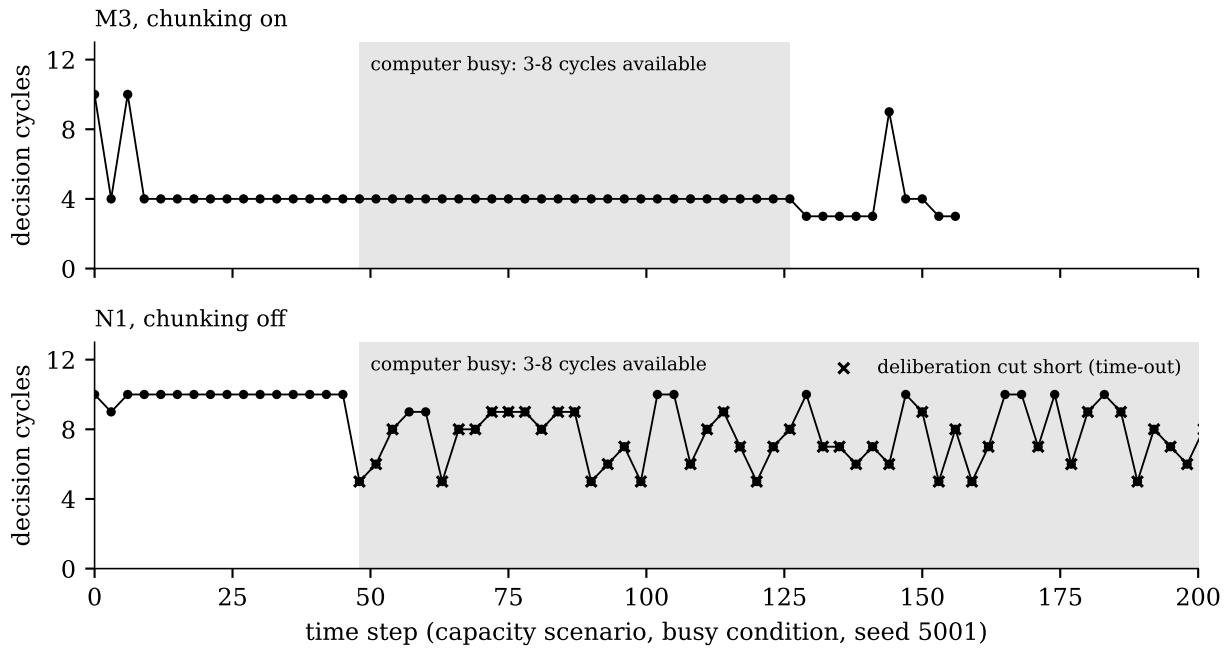


Figure 2: Decision cycles used per robot decision over the first 200 steps (capacity scenario, busy computer, seed 5001). Grey band: the computer is busy. Crosses: deliberations cut short by the time limit.

How to read Figure 2. With chunking (top), decisions settle at 4 cycles after the first few, below every busy budget, so no deliberation is ever cut short. Without chunking (bottom), decisions need 9–10 cycles; as soon as the computer is busy, they are capped at the budget (3–8) and many are cut short.

12 Results for M3

12.1 The hypotheses

		Seeds 1–200		Seeds 20001–20200	
	Prediction	Mean [CI]	p_{Holm}	Mean [CI]	p_{Holm}
H4b	no shift (quiet − free < +0.05)	0.000 [0.000, 0.000]	<.001	0.000 [0.000, 0.000]	<.001
H4c	no capacity effect (±0.05), chunking on	0.000 [0.000, 0.000]	<.001	0.000 [0.000, 0.000]	<.001
H4d	capacity effect without chunking	0.838 [0.810, 0.865]	<.001	0.849 [0.824, 0.872]	<.001

Table 8: H4b–H4d (Holm correction over the three tests).

H4a, no anticipation effect (structural). The two anticipation conditions differ only in Clarion's achievement stimulus, which M3 does not represent. So for M3 the two conditions are the same experiment, and all 200 seeds gave identical episodes. This is a consequence of the design, not a discovery, and is reported as such.

H4b, no motivational shift. The lapse rate after room A is 0 in both shift conditions: the difference is exactly 0, below the margin of +0.05.

H4c, no capacity effect with chunking. Lapse rates while busy are 0 with a busy and with a free computer: equivalent within ±0.05.

H4d, a capacity effect without chunking. Without chunking, the lapse rate while busy is 0.84 higher with a busy computer. This is the largest effect in the study ($d_z \approx 4$).

Every M3 hypothesis is supported on both sets of seeds.

12.2 Every variant

					Time-outs per episode		Lapse rate while busy	
Variant	Lapse rate	Room C %	Steps	Planning calls	free	busy	free	busy
M3, full model	0.000	0.0	165	49.5	0.0	0.0	0.000	0.000
baseline: habit only	1.000	100.0	300	0.0	0.0	0.0	1.000	1.000
baseline: always deliberate	0.000	0.0	165	55.9	0.0	0.0	0.000	0.000
N1 no chunking	0.136	15.2	196	59.4	0.0	70.1	0.000	0.838

Table 9: M3 and its variants (seeds 1–200; time-outs and lapse while busy: capacity scenario).

- M3 never lapses and never breaks the quiet-zone rule, and it finishes in fewer steps than CLARION (165 against 198). Its control is **all-or-none**: once the task knowledge resolves a conflict, Soar's symbolic preferences always override the habit.
- *Always deliberate* behaves like M3 but uses more planning calls: deliberating where there is no conflict adds cost without benefit.
- *Habit only* lapses at every exposure: the learned habit alone is a pure temptation-seeker.
- *N1* lapses only when time runs out, which happens only while the computer is busy.

12.3 What M3 shows

In native Soar, self-control depends on **knowledge and time**, not on motivation. When the robot knows that an option is off-task and has time to apply that knowledge, control is complete. Motivation-based effects (anticipation, motivational shift) cannot arise, because nothing in the model changes with them. Capacity can matter, but only for conflicts that have not been compiled: Soar predicts that *practice protects self-control from cognitive load.*

13 Open design questions for M3

1. Replace the hand-written task values with Soar's default selection space and look-ahead evaluation.
2. Let Soar-RL learn when to deliberate (a learned value for `deliberate`). This could give graded, context-dependent control in native Soar, and would be the strongest test of whether CLARION's effects require drives.
3. Use appraisal-based emotion (Marinier et al., 2009) to make a choice point salient: would that produce an anticipation effect?

4. Replace the special cases with a general rule: an option is off-task if it does not advance an assigned goal.
5. Sweep the busy budget (2–10 cycles) to find where a capacity effect appears even with chunking.

References

Laird, J. E. (2012). *The Soar cognitive architecture*. MIT Press.

Marinier, R. P., Laird, J. E., & Lewis, R. L. (2009). A computational unification of cognitive behavior and emotion. *Cognitive Systems Research, 10*(1), 48–69.